(?) 1. 1. Vermischtes zur Einführung 2015

Markieren Sie alle zutreffenden Aussagen:

- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: *{ x = y; y = r; r = x % y; }*
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- *public static void main(String[] arks)* ist Kopf der *main* -Methode.

(?) 1. 1. Allgemeine Grundkenntnisse 2016

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise präzise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist. (JVM muss installiert sein)
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine Pipe funktioniert im wesentlichen wie eine Queue.
3. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung kann man zerlegen in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 7 Hexadezimalziffern kann man mehr als 1 Milliarde Werte darstellen. 268 milionen
   - (D) Der Bruch 4/5=0.8 kann exakt als double-Wert dargestellt werden.
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft ihn auf syntaktische Korrektheit.

(?) 1. 1. Allgemeine Grundkenntnisse (I1-ID: mq4g9vq04ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - A) Eine Spezifikation ist eine präzise Beschreibung der erlaubten Ausgaben.
   - (B) Algorithmen beschreiben Handlungsvorschriften (z.B. zur Berechnung eines Werts).
   - (C) Programmentwicklung heißt (immer wieder testen, dann compilieren, dann editieren.
   - (D) Compiler laden Maschinencode zusammen mit Klassenbibliotheken in das Laufzeitsystem.
2. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Der Befehlszähler im von Neumann-Prozessormodell speichert arithmetische Befehle.
   - (B) Interaktive Programme können auf Daten von der Standard-Eingabe reagieren. -
   - (C) Die Pipe der bash-Shell ist der Stack der noch nicht verarbeiteten Eingaben.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Durch Ausgabeumleitung kann man erzeugte Daten in einer Datei abspeichern.
   - (B) Decodieren bedeutet, eine Nachricht als Information zu interpretieren bzw. zu verstehen.
   - (C) Durch N Hexadezimalziffern werden 4mal soviel Informationen dargestellt wie mit N Binärziffern.
   - (D) Der ASCII-Standard wurde zur Lösung des Jahr-2000-Problems durch Unicode ersetzt.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Strukturierte Daten sind zerlegbar in kleinere Bestandteile.
   - (B) Gleitkomma-Arithmetik ist schneller als Ganzzahl-Arithmetik.
   - (C) Die JVM interpretiert Bytecodebefehle ohne ihre Syntax zu prüfen.
   - (D) Imperative Programmierung heißt, Daten durch Befehle zu ersetzen.

2. 1. Zweierpotenzen 2015

Ergänzen Sie untenstehende *main*-Methode so, dass der Aufruf *java Zweierpotenzen*
die Liste der Zweierpotenzen von *2^0 = 1* bis *2^n* zeilenweise ausgibt

```
public class Zweierpotenzen {
        public static void main (String[] args) {
        int n = Integer.parseInt(args[0]);
        int p = 1;
        for(int i = 0; i < n; i++){
                System.out.println(p);
                p = p*2;
        }
}
```

Beispiel *java Zweierpotenzen 3* gibt diese Liste aus:
1
2
4
8

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):
initialisiere:
- Kommandozeilenargument (interpretiert als int -Wert) in n speichern
- int-Variable p als Akkumulator initialisieren

iteriere so oft wie nötig:
- p ausgeben und geeignet aktualisieren

finalisiere: entfällt

Probeklausur 2019 Punktklasse

Ein Punkt der Ebene soll allein durch seine Koordinaten gegeben sein. Schreiben Sie eine Java-Klasse für *unveränderliche* Punkt-Objekte, die folgende API realisiert. Alle Variablen sind private zu deklarieren, Instanzvariablen auch final.
- public class Punkt
  - public Punkt(double x, double y)// Konstruktor
  - public static int anzahl() // liefert die Anzahl der erzeugten Punkte
  - public double getX() // liefert x-koordinate des Punktes
  - public double getY() // liefert y-koordinate des Punktes
  - public String toString()// liefert eine String-Darstellung: “Punkt(...,....)
    aus dem man die Koordinatenwerte ablesen kann

2. 1. Einige Java-Grundkenntnisse 2016

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)
   Algorithmus, Compiler, Editor, Loesungsidee, Quelltext, Spezifikation, Testen, Zielcode

Spezifikation, LoesungsIdee, Algorithmus, Editor, Quelltext, Compiler, Zielcode, Testen

2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es wird die Summe der ganzen Zahlen ausgeben, die innerhalb des abgeschlossenen Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
        public static void main(String[] args) {
                A ;
                min = B  (args[0]);
                max = C (args[1]);
                int D ;
                while( E ){
                        F;
                        G;
                }
                H;
        }
}
```

A: double s = A, int s = A, double s = 0; int s = 0
B: StdIn.readInt, Integer.parseInt
C: StdIn.readInt, Integer.parseInt
D: i = min, n = min, t = min
E: min <= n, min <= s; n <= max, s <= max
F: n += s, s+=n, n=n+1, s = s+1
G: n++, s++, n+=s, s+=n
H: return n, return s, System.out.println(n), System.out.println(s)

2. Wieviele Bytes beansprucht eine Variable?

- int: 4
- char: 2
- float: *4*
- boolean: 1

1, 2,4,8,16 , nichts von denen.
(0.25P für jede richtige Antwort)

(?) 2. 1. Einige Java-Grundkenntnisse (l1-ID: ia4a8tq09ih0)

1. Von klein nach groß ordnen (0.25P für jede korrekte Angabe)
Was kann ineinander enthalten bzw. Teil voneinander sein?
Ordnen Sie von OBEN nach UNTEN die Begriffe vom "kleinsten" bis zum "größten", d.h. wenn X Teil von Y sein kann, so muss X weiter oben stehen als Y:

Anweisung; Ausdruck; Datei; Java-Archiv; Klasse; Klassenpfad; Literal; Methode
Literal
Ausdruck
Anweisung
Methode
Klasse
Datei
Java-Archiv
Klassenpfad

3. 1. Quadratzahlentabelle 2015

Die *int*-Variable *n* speichere die ganze Zahl *n*.
Die ersten *n* positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile ausgegeben werden.
Beispiel für *n = 4* :
1 : 1
2 : 4
3 : 9
4 : 16
Im Fall *n <= 0* soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer *for*-Schleife und einmal mit einer *do-while*-Schleife.

Achten Sie auf den Unterschied zwischen *System.out.print(..)* und *System.out.println(..)* .

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
        System.out.print(i + ": ");
        System.out.println( i*i);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
        System.out.print(i + ": ");
        System.out.println( i*i);
        i++;
} while ( i < n+1);
```

(?) 4. 2. Vermischtes zu elementaren Daten 2015

Markieren Sie **alle** zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- *int*-Werte können verlustfrei in *float*-Variablen gespeichert werden.
- *5/4 == 1* hat den Wert *true*.
- *Integer.MAX_VALUE*++ verursacht keinen Ganzzahlüberlauf.
- Anstatt *if (b > 0 && a/b < 2)* .. schreibt man besser *if (a/b < 2 && b > 0)* ...
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik 2015 2016

Sei *d* eine *int*-Variable mit dem Wert *Integer.MAX_VALUE* (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

1: System.out.println(-d);	*-2147483647*
2: System.out.println(d-1);	*2147483646*
*3: System.out.println(1+2*d);*	*-1*
*4: System.out.println(-(2*d));*	*2*
5: System.out.println(2147483646-d);	-1
6: System.out.println((-1)*d);	*-2147483647*
7: System.out.println(d+2);	*-2147483647*
8:System.out.println(d*2-1);	*-3*

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0) *2015*

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
        char a = 'a';
        a++;
        int b = (int) a;
        b++;
        System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
        System.out.print("T");
        return true;
}
public static boolean getFalse() {
        System.out.print("F");
        return true;
}
public static void q(int i) {
        int j = 43;
        System.out.print("G");
        if ( i / j > 0) System.out.print("Q");
        else System.out.print("R");
}
public static void d() {
        double dd = Double.MIN_VALUE;
        dd *= 2;
        dd -= Double.MIN_VALUE;
        dd /= 2;
        System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der *main*-Methode der gleichen Klasse)?

- *a();* — *B*
- *boolean val = getTrue() || getFalse();* — *T (wenn beim logischen oder das erste stimmt wird das zweite nicht mehr ausgefuehrt)*
- *val = getTrue() ^ getTrue();* — *TT (exlusives oder)*
- *System.out.print(val);* — false
- *q(42);* — *GR*
- *d();* — *0.0*

3. 2. Grundlegende Datentypen 2016

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) 'C'++ liefert das gleiche Ergebnis wie 'C'+1.
   - (D) "C" + 1 ist kein erlaubter Java-Ausdruck.

2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) int ist ein Ganzzahldatentyp, char nicht.
   - (B) int x = Integer.MAX_VALUE + 1; ist eine erlaubte Anweisung.
   - (C) Der Wert von (3 < 7) && ((17 + 4) % (12 - 5)) != 0) ist false.
   - (D) Der Wert von (3 + 1/3 == 3 - 1/3) ist false.

Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Das Tabulatorzeichen ist ein Steuerzeichen.
   - (B) Unicode ist eine Teilmenge von iso8859-1.
   - (C) Der Wert von (7 % (5 - 2) == 0) || (7 % (5 + 2) == 0) ist true.
   - (D) Der Wert von (1000 + 11/12 == 1000 - 11/12) ist false.

3. Wählen Sie die zutreffenden Antworten aus:
   - (A) String s = "Hallo"; System.out.println(s.length);
     Ausgabe?

     (5,6,keine(wegen Syntaxfehler), keine (wegen logischem Fehler)) s.length() waere richtig

   - (B) System.out.println(10e-1f);
     Ausgabe?

     (1,1.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler)) 10 verschoben um -1 nach hinten

   - (C) 2 * (short) 0xCafe
     Typ des Ausdrucks?

     (short,int, undefiniert (wegen Syntaxfehler), undefiniert (wegen logischem Fehler))

   - (D) double y = Double.MIN_VALUE; System.out.println(y/2);
     Ausgabe?

     (0,0.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler))

   - (E) int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;
     Wert von z?

     (-1,+1, undefiniert (wegen Syntaxfehler), undefiniert (wegen logischem Fehler))

   - (F) System.out.println((double) (1/2));
     Ausgabe?

   - (0,0.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler))

4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) int[] z = {1}; System.out.println(z); produziert die Ausgabe 1.
   - (B) length ist ein private-Attribut von Feldern.
   - (C) Durch int[] a = {1, 2, 3}, b = new int[]; werden zwei neue Felder angelegt.
   - (D) Durch int[][][][] x = new char[32][32][32][16]; wird etwa 1Mb Heap-Speicher reserviert.(Stack) (1MB 1000000 B) 1048576
   - (E) int[] z = {1}; System.out.println(z.length()); produziert die Ausgabe 1.

7. 3. Rekursion 2015

Betrachten Sie folgende Klasse:

```
1:  class Myst {
2:          static void ery(int n) {
3:                  if (n == 0)
4:                          return;
5:                  ery( n-1);
6:                  A();
7:                  yre( n-1);
8:          }
9:          static void yre(int n) {
10:                 if (n == 0)
11:                         return;
12:                 ery( n-1);
13:                 B();
14:                 yre( n-1);
15:         }
16:         static void A() {
17:                 System.out.println("A");
18:         }
19:         static void B() {
20:                 System.out.println("B");
21:         }
22:         public static void main(String[] args) {
23:                 int n = Integer.parseInt(args[0]);
24:                 ery(n);
25:                 System.out.println();
26:         }
27:}
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf *java Myst 4* an:

- AABAABAA
- BBABBAAB
- ABAAABBA
- BABBBAAB
- AABAABBA
- BBABBAAB
- AABBAABA
- BBAABBAB
- AABABABB
- BBABABAA

2. Aus wie vielen Zeichen besteht der Ausgabestring beim Aufruf *java Myst 7* (ohne den abschließenden Zeilenwechsel durch Zeile 25)?

15, 16, 17, 49, 50, 51, 127, 128, 129, 255, 256, 257

3. Wird Zeile 14 durch *yre(n-2);* ersetzt, so führt beispielsweise der Aufruf *java Myst 4* zu einem Fehler.
Welche andere Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

Zeile 3, Zeile 4, Zeile 5, Zeile 6, Zeile 7, Zeile 10, Zeile 11, Zeile 12, Zeile 13,
(zu kleiner gleich)

4. Werde Zeile 12 durch *ery(n-2);* und zusätzlich Zeile 14 durch *yre(n-2)* ersetzt.
Geben Sie das maximale Argument n an, bei dem der Aufruf *java Myst n* nicht zu einem Fehler führt:

-3, -2, -1, 0, 1, 2, 3

7. 3. Statische Methoden (l1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
        public static void q1(int n) {
                n = n * n;
        }
        public static void q2(int[] f, int i) {
                f[i] = f[i] * f[i];
        }
        public static void main(String[] args) {
                int n = 2;
                int[] f = { n, n, n};
                for (int i = 0; i < 3; i++) {
                        q1(n); q2(f,n);
                }
                System.out.println(n);
                System.out.println(f[n]);
        }
}
```

Wie lautet die erste Ausgabezeile?

2

Wie lautet die zweite Ausgabezeile?

256

8. 3. Rekursion (l1-ID: 4mqjzge16ih0) 2017

```
public static void strange(int n) {
        if ( n <= 2) return;
        System.out.print("(");
        strange(n-2);
        strange(n-3);
        System.out.print(")");
}
```

Welche Ausgabe bewirkt der Aufruf strange(7)? Tipp: Rekursionsbaum benutzen

((())())

9. 4. Felder: Fehler oder ...? 2015

Erzeugt untenstehende Klasse

- einen Compilezeitfehler?Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein,ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich: 0

```
1: class FehlerOderWie {
2:          public static void main(String[] args) {
3:                  int[] a = {2,3,5,7,11};
4:                  int[] b = new int[2];
5:                  b = a;
6:                  b[4] = -4;
7:                  if (a[4] >= 1) System.out.println(1);
8:                  else System.out.println(0);
9:          }
10: }
```

6. 4. Felder: Fehler oder...?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler?Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich: -
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich : false

```
1: class FehlerOderKeiner {
2:          public static void main(String[] args) {
```

```
3:                  int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:                  b = a;
5:                  b[1] = b[0];
6:                  System.out.println(a[0] != a[1]);
7:          }
8: }
```

11. 4. Iteration in einem Feld 2015

Tragen Sie die Werte der Feldkomponenten nach Ausführung der for-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
for (int i = 0; i < 10; i++)
        d[i] = d[9-d[9-d[9-i]]];
```

vorher 0 1 2 3 4 5 6 7 8 9
nachher 9 8 7 6 5 5 6 7 8 9

5. 4. Iteration in Feldern

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
        c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
        q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten nach Ausführung des Java-Fragments ein: Index

index	0	1	2	3	4	5	6	7
c	1	1	1	1	1	1	1	1
q	2	0	5	3	1	7	4	6

5. 4. Iteration in Feldern (I1-ID: 435dkh80w5h0)

```
int[] p = { 1, 0, 7, 3, 5, 4, 6, 2 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
        c[i] = p[p[i]];
for (int i = 0; i < 8; i++)
        q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten nach Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
c	0	1	2	3	4	5	6	7
q	1	0	7	3	5	4	6	2

Aufgabe 3

Betrachten Sie folgende Klassen und beantworten Sie darunterstehenden Fragen ohne einen Computer zu Hilfe zu nehmen.

```
1: public class Katze {
2:    private String name;
3:    public Katze(String name) {
4:       this.name = name;
5:    }
6:    public String miezmiez() {
7:       return "Fauch!";
8:    }
9: }
1: public class Hauskatze extends Katze {
2:    private String name;
3:    public Hauskatze(String name) {
4:       super(name);
5:    }
6:    public String miezmiez() {
7:       return "Miau Miau";
```

```
8:    }
9: }
1: public class Tierfreund {
 2:    public static void main(String[] args) {
 3:       Hauskatze h = new Hauskatze("Schnurri");
 4:       Katze k = new Katze("Garfield");
 5:       System.out.println(((Katze)h).miezmiez());
 6:       System.out.println(((Hauskatze)k).miezmiez());
 7:       k = h;
 8:       System.out.println(((Hauskatze)k).miezmiez());
 9:    }
10: }
```

- In Tierfreund ist eine Zeile fehlerhaft. Welche Zeile ist es?
  Zeile 6
- Geben Sie an, welche Art Fehler die Zeile bewirkt (Compile- oder Laufzeitfehler).
  Laufzeit

Welche Ausgabe erzeugt java Tierfreund wenn die fehlerhafte Zeile gestrichen wird?

Miau Miau
Miau Miau

16. 5. OOP-Entwurf 2015

Die Klasse *Ente* erbt von der Klasse *Vogel*. In *Vogel* gibt es eine Instanzmethode *fliege(..)*, in Ente gibt es die Instanzmethode schnattern().
Die fliege-Methode von Ente-Objekten soll nun so angepasst werden, dass schnattern() immer unmittelbar am Anfang von fliege aufgerufen wird.
Wie ist das am besten zu realisieren?

```
public void fliege(Ziel z) {
        this.schnattern();
        // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
public void fliege(Ziel z) {
        this.schnattern();
        super.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
        this.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
        Vogel.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
}
```

17. 5. Einfach verkettete Listen 2015

x sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

- Knoten t wird direkt hinter Knoten x eingefügt.

(?) 11. 5. OOP-Entwurf

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:

class Aktuer

class GuterKaempfer extends Aktuer implemnts Kaempfer
class BoeserKaempfer extends Aktuer
class GefaehrlicheKreatur extends Aktuer
class FreundlicheKreatur extends Aktuer

Die (hier nichtt gezeigte) Klasse Spiel benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie wird folgende Zusatzanforderung korrekt modelliert? Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode *benutzeWaffe()* ausgestattet werden.

- *benutzeWaffe()* in *Akteur* implementieren, in *GefaerlicheKreatur* und *FreundlicheKreatur* mit *private* kennzeichnen.
- Interface Kaempfer mit benutzeWaffe() definieren und durch GuterKaempfer und BoeserKaempfer implementieren lassen.
- Die Methode *benutzeWaffe()* nur in GuterKaempfer und in BoeserKaempfer wie benötigt implementieren.
  Weitere Klasse Kaempfer schreiben, die benutzeWaffe() implementiert, GuterKaempfer, BoeserKaempfer davon erben lassen.
- benutzeWaffe() mit leerem Rumpf in Akteur implementieren, in GuterKaempfer und BoeserKaempfer geeignet überschreiben.

15. 5. OOP-Verständnisfragen 2015 2016
Sei p eine Referenzvariable vom Typ X. Was kann alles der dynamische Typ von p sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Antworten aus (jeweils 1.5P für die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von X (falls X eine Klasse ist)
- (B) ein elementarer Datentyp (falls X die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die X implementiert (falls X ein Interface ist)
- (D) eine Klasse, von der X direkt oder indirekt erbt (falls X eine Klasse ist)
- (E) ein Interface, das X als abstrakte Klasse erweitert (falls X eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die X implementiert (falls X ein Interface ist)

12. 5. OOP-Verständnisfragen (I1-ID: c6j5qfl08ih0) 2017
Frage 1 Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?

Durch public/private/protected Instanzvariablen und public/private/protected Methoden.

Frage 2 Welche der Konzepte (A) Polymorphie, (B) Vererbung,(C) Kapselung, (D) Überladen kann man anhand folgender Situationsbeschreibung illustrieren?
Jedes Tier kann einen Laut von sich geben (ein Geräusch machen). Jeder Hund ist ein Tier und kann einen Laut von sich geben, nämlich: Wau Wau! Jede Katze ist ein Tier und kann einen Laut von sich geben, nämlich: Miau!

10. 5. Vermischtes zu Klassentypen 2015
1. Objekte verstehen:

```
public class Spieler {
        private int score;
        public Spieler( int s) {
                this.score = s;
        }
}
```

Zwei Spieler-Objekte werden instanziiert:
Spieler a = new Spieler(50), b = new Spieler(50);
Welchen Wert hat nun der Ausdruck a == b?

- true, false, das ist Compile-anhaengig, das ist speicherabhaengig, nichts von denen

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:          private hepno.users.Wring wrags;
3:          public FirstBpp() {
```

```
4:                      wrags = new hepno.users.Wring();
5:          }
6:          public static void main(String[] args) {
7:                      FirstBpp app = new FirstBpp();
8:          }
9:  }
```

Was bewirkt Zeile 2?:

- auf hepno.users.Wring kann mit wrags zugegriffen werden
- wrags wird als Instazvariable vereinbart
- hepno.users.Wring wird als private-Objekt deklariert
- wrags instanziiert die Klasse hepno.users.Wring
- nichts von denen

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
        private final int zahl = 10;
        double a() {
                System.out.println(zahl);
                return c();
        }
        void b() {
                System.out.println(this);
        }
        double c() {
                double r = Math.random();
                System.out.println(r);
                return r;
        }
        void d() {
                a();
                a();
        }
        int e() {
                return zahl;
        }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit static versehen werden)?

- a, b, c, d, e

4. Fehlersuche:

```
public class Test {
        private int testCount;
        public static int getCount(){
                return testCount;
        }
        public Test(){
                testCount++;
        }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

- testCount wurde nicht initialisiert
- auf testCount wurde nicht zugegriffen
- getCount() ist statisch
- testCount ist private
- Test() hat keinen Ergebnistyp
- nichts von denen

5. Tracing:

[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann

auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
        static int tsetCount = 0;
        public Tset(){
                tsetCount++;
        }
        public static void main(String [] Args){
                // Hinweis: auf Klassenvariablen kann man auch
                Tset t1 = new Tset();
                System.out.print(t1.tsetCount + ", ");
                Tset t2 = new Tset();
                System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
                Tset t3 = new Tset();
                System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
                System.out.println();
        }
}
```

Welche Ausgabe produziert der Code?

- 0, 0 0, 0 0 0
- 1, 1 1, 1 1 1
- 1, 2 2, 3 3 3
- 1, 2 3, 4 5 6
- nichts von denen

6. Begriffsverständnis:

Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

- In einer Klasse darf es Methoden geben, die den gleichen Namen haben wie ein Konstruktior
- In einer Klasse darf es Methoden geben mit gleichem Namen und gleichen Ergebnistyben, aber unterschiedlich vielen Argumenten
- In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.
- In einer Klasse darf es mehrere Konstruktoren mit den gleichen Namen geben

14. 5. Eine Bibliothek für Permutationen (I1-ID: 8834zrd08ih0) 2017

Definition

Permutationen entsprechen int-Feldern einer Länge n, die die Werte 0,1,...,n-1 speichern (in beliebiger Reihenfolge).
Das Quadrat der Permutation p, ist die Permutation q mit: q[i] ist gleich p[p[i]] für alle Indexwerte i..
Die Unordnung der Permutation p ist die Anzahl der Indexpaare (i,j) mit i < j und p[i] > p[j]. Zum Beispiel hat (2,0,1) Unordnung 2..

Aufgabe

Schreiben Sie eine Klasse Permutation mit den statischen Methoden

- quadrat(..) und unordnung(..), die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:
- 1. quadrat(p) ermittelt und liefert das Quadrat der Permutation p
- 2. unordnung(p) ermittelt und liefert die Unordnung von p.

Es ist jeweils gesichert, dass p wirklich eine Permutation ist - nicht separat prüfen!

```
public static int[] quadrat(int[] p){
        int[] q = new int[p.lenght];
        for(int i = 0; i < p.lenght; i++){
                q[i] = p[p[i]];
        }
        return q;
}

public static int unordnung(int[] p){
        int unordnung = 0;
        for(int i = 0; i < p.length; i++){
                for(int j = 0; j < p.length; j++){
```

```
                        if(i<j && p[i] > p[j]){
                                unordnung++;
                        }
                }
        }
        return unordnung;
}

public static boolean check(p){
        for(nt i = 0; i < p.lenght; i++){
                if(p[i] < 0 || p[i] >= p.lenght){
                        return false;
                }
        }
        int[] c = new int[p.lenght];
        for(int i = 0; i < p.lenght; i++){
                if(c[p[i]] == 1){
                        return false;
                }
                c[p[i]] = 1;
        }
        return true;
}

public static int[] invers(int[] p){
        int q = new int[p.length];
        for(int i = 0; i < p.lenght; i++){
                q[p[i]] = i;
        }
}
```

14. 5. Eine Bibliothek für Permutationen

Definition

1. Wir nennen ein int-Feld p der Länge n eine Permutation, wenn p[0], …, p[n-1] im Bereich liegen und jeder Wert genau einmal vorkommt.
2. Die Permutation q heißt Inverse der Permutation p, wenn für alle i der Wert von q[p[i]] gleich i ist.

Aufgabe

Schreiben Sie eine Klasse Permutation mit den statischen Methoden check(..) und invers(..), die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

- check(p) überprüft, ob p eine Permutation ist und liefert das Ergebnis (Hinweis: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfüllt ist, dann mit zweiter Schleife in einem Hilfsfeld c die Vorkommen der Werte p[0], …, p[n-1] erfassen)
- invers(p) ermittelt und liefert die Inverse von p. (Dabei ist bereits gesichert, dass p wirklich eine Permutation ist - nicht separat prüfen! Hinweis: Feld anlegen und in Schleife die Komponenten so setzen, wie es Definition 2. vorschreibt.)

15. 5. Eine einfache Klasse (I1-ID: 3g3j2i70x5h0) 2017

Programmieren Sie eine einfache Klasse Wein mit der Angaben zu Weinflaschen abgespeichert werden können.
Jeder Wein zeichnet sich durch seine unveränderliche Sorte aus (z.B. "2001er Goldtropfen"), und durch seinen Preis (z.B. 13,50€).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (wenn nicht anders angegeben: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer Wein-Instanz unter Angabe der Sorte
- Erzeugen einer Wein-Instanz unter Angabe der Sorte und des Preises
- Setzen des Preises einer Instanz (nur von Wein-Methoden aus),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe ihrer Preise.

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER: sorte, preis, anzahl, gesamt Zum Einrücken bitte unbedingt die Tabulatortaste anstelle Leerzeichen benutzen (siehe unten).

```
public Wein{
	private  final String sorte;
	private int preis;
	private static int anzahl = 0;
	private static int gesamt = 0;

	public Wein(String sorte){
		this.sorte = sorte;
		anzahl++;
	}
	public Wein(String sorte, int preis){
		this.sorte = sorte;
		this.preis = preis;
		anzahl++;
		gesamt += preis;
	}
	private void setPreis(int preis){
		this.preis = preis;
	}
	public getPreis(){
		return preis;
	}
	public static int getAnzahl(){
		return anzahl;
	}
	public static int getGesamt(){
		return gesamt;
	}
}
```

(?) 15. 5. Eine einfache Klasse 2016

Programmieren Sie eine einfache Klasse Auto mit der Angaben zu Autos abgespeichert werden können.
Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (wenn nicht anders angegeben: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer Auto-Instanz unter Angabe des Typs, der Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein
- Erzeugen einer Auto-Instanz unter Angabe des Typs und des Preises,
- Setzen des Preises einer Instanz (nur von Auto-Methoden aus),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER: *typ, preis, anzahl, gesamt*

(?) 18. 5. Eine einfache Klasse 2015

Programmieren Sie eine einfache Klasse *Buch* mit der Angaben zu Büchern abgespeichert werden können. Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer *Buch*-Instanz unter Angabe des Titels
- Erzeugen einer *Buch*-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER: *titel, seiten, anzahl, gesamt*
Außerdem soll die Klasse für einen ersten Test mit einer *main* -Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein *Buch*-Objekt erzeugen nur unter Angabe des Titels *Lieblingsbuch* und die Referenz der Variablen *b* zuweisen

- anschließend die Seitenzahl dieses Buchobjekts auf *500* setzen
- schließlich mittels *b* die Angaben zu dem erzeugten Objekt ausgeben: *Lieblingsbuch, 500 S.*

(?) 14. 6. Vermischtes zu Algorithmen und Datenstrukturen 2015

Markieren Sie alle zutreffenden Aussagen:

- Binäre Suche erfordert O(n) Vergleiche auf **unsortierten** Feldern.
- Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- Quicksort ist auf vorsortierten Feldern besonders schnell.
- Mergesort benötigt auf allen Feldern der Länge 2^m immer di\e gleiche Anzahl an Vergleichen.
- Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Binäre Bäume der Tiefe n haben mindestens n - 1 und höchstens 2^n - 1 innere Knoten. (geht mit n+1 statt n)
- Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

9. 6. Binäre Suche protokollieren (l1-ID: i5k29e615ih0) 2017

Betrachten Sie folgende Implementierung der binären Suche in einem aufsteigend sortierten Feld:

```
public static int binarysearch(int[] f, int q) {
int lo = 0, hi = f.length - 1;
while (lo<=hi) {
        int m=(hi+lo)/2;
        if (q<f[m]) {
                System.out.print("L");
                hi = m - 1;
        }
        else if ( q>f[m] ){
                        System.out.print("R");
                        lo=m+1;}
                else {
                System.out.print("!");
                return m;
                }

}
System.out.print("-");
return -1;
}
```

Hier ein Ausschnitt aus dem Unit-Test.

```
int[] g = { 1, 1, 3, 5,    6, 7,   8, 9 ,   9, 9, 10, 11, 11, 12, 13, 15};
int s = g[4];
System.out.println(binarysearch(g,s));
```

Welche Ausgabe wird erzeugt?

LRL!4

11. 6. Einen Stack kopieren (l1-ID: kw3g7f70y6h0) 2017

Betrachten Sie folgende API von Stacks zum Speichern von *int*-Werten (alle Methoden/Konstruktoren *public)*:

public class Stack

- *Stack()* erzeugt einen neuen Stack
- boolean empty() testet, ob dieser Stack leer ist
- void push(int i) speichert i in diesem Stack
- int pop() in diesem Stack zuletzt gespeicherter Wert wird geliefert und entfernt

Stellen Sie die Zeilen für den Rumpf einer weiteren Instanzmethode

```
public Stack copy() {
        Stack h = new Stack();
        Stack c = new Stack();
        while (!this.empty()) {
                h.push( this.pop()); }
        while (!h.empty()) {
                int i = h.pop();
                this.push(i);
                c.push(i);}
        return c;
}
```

in richtiger Reihenfolge zusammen, so dass ihr Aufruf einen Stack mit gleichem Inhalt wie dieser liefert (d.h. es soll dann möglich sein, Original und Kopie unabhängig zu benutzen: Einspeichern beim Original ändert nicht den Zustand der Kopie usw.).

Idee: (1) alle Werte aus diesem Stack zunächst aus- und in einen neuen Stack h einspeichern, dann (2) von dort alle Werte aus- und sowohl in diesen Stack als auch in neuen Stack c einspeichern, schließlich Referenz der Kopie liefern.

(?) 13. 6. Analyse von Algorithmen 2016 (diese Aufgabe ist nicht Klausurrelevant denke ich)

1.Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:         for (int j = i+1; j < N; j++)
3:                 for (int k = j+1; k < N; k++)
4:                         verarbeite(daten);
```

Die Ausführungszahl *V(N)* von Zeile 4 bei Problemgröße *N* soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere asymptotische Analyse geeignet?

N(N-1)*(N-2), N*(N-1)*(N-2)/3 , N*(N-1)*(N-2)/6 , ~N^3, ~(n^3)/3 ~(N^3)/6, O(N^3), O((N^3)/3), O((N^3)/6)*

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehen typischerweise so aus:

Problemgröße N:	*102*	*2048*	*4096*	*8192*	*16384*	*32768*	*65536*	*131072*	*62144*	*524288*	*1048576*	*2097152*	*4194304*	*8388608*
Zeit (s) :	*0.00*	*0.01*	*0.01*	*0.02*	*0.03*	*0.06*	*0.14*	*0.30*	*0.65*	*1.24*	*2.46*	*4.98*	*10.02*	*19.94*

Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße wächst. Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

- *worst-case quadratisch und best-case logarithmisch*
- *average-case linear und best-case quadratisch*

- *best-case linear und average-case quadratisch*
- *average-case quadratisch und worst-case linear*
- *average-case linear und best-case linear*
- *worst-case quadratisch und average-case quadratisch*
- *best-case logarithmisch und worst-case logarithmisch*
- *average-case quadratisch und worst-case quadratische*

-- 3. Welche dieser Aussagen trifft NICHT zu?

- *Bubblesort ist quadratisch im worst case*
- *Insertion- und Selection-sort haben asymptotisch gleiche worst-case Laufzeit*
  Binaere Suche erfordert sortierte Eingabe
- *Quicksort hat auf sortierter Eingabe lineare Laufzeit*
- *Quicksort ist ein in-place Sortierverfahren*
- *Vergleichsanzahl bei Selection-sort hängt nur von Feldlänge ab*
- *vergleichsbasierte Sortierverfahren haben mindesten linearlogariithmisches worst-case-Verhalten*

12. 6. Stack und Queue in Aktion

1. Auf einem Stack werden in gemischter Reihenfolge 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.
Welche Ausgabelisten sind möglich?
Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

- (A) 1 2 3 4 5 6 9 8 7 0
- (B) 0 4 6 5 3 8 1 7 2 9
- (C) 1 4 7 9 8 6 5 3 0 2
- (D) 2 1 4 3 6 5 8 7 9 0

2. Auf einer Queue werden in gemischter Reihenfolge 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich? Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

- (E) 4 6 8 7 5 3 2 9 0 1
- (F) 2 5 6 7 4 8 9 3 1 0
- (G) 0 1 2 3 4 5 6 7 8 9
- (H) 4 3 2 1 0 5 6 7 8 9

(?) 9. 6. Queue aus zwei Stacks bauen 2016

Mit zwei Stacks kann man eine Queue implementieren:

- enqueue(...) entspricht Einspeichern in inbox
- dequeue() entspricht Ausspeichern aus outbox.

Unter Verwendung dieser API
public class Stack

- boolean empty() testet, ob dieser Stack leer ist
- void push(String s) speichert s in diesem Stack

- String pop() liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

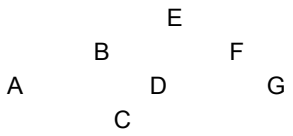
```
public class ZSQueue { // Zwei-Stack-Queue
        private Stack inbox, outbox;
        public ZSQueue() { ... }
        public boolean empty() { ... } // Leerheitstest
        public void enqueue(String s) { ... } // einspeichern
        public String dequeue() { // ausspeichern und liefern
                // Rumpf (Ihre Aufgabe)
        }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode dequeue() richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob outbox leer ist und fülle in diesem Fall den kompletten Inhalt von inbox um in outbox, dann ein Element aus outbox ausspeichern und liefern.

13. 6. Binärer Suchbaum 2015

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
Welcher ist das?

Inorder ABCDEFG
preorder (erst wurzel dann linker dann rechter Teilbaum) EBADCFG
postorder(erst linker dann echter teilbaum und dann wurzel) ACDBGFE

F

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:
DBFACEG